

Deployment Guide

Step-by-step instructions for taking the Shopster MERN application live — the Express API on **Render** and the React + Vite storefront on **Vercel**. Each part is self-contained, but the two must be wired to each other at the end.

PART A

Jeet & Vikram

Backend — Express 5 + MongoDB + Stripe, deployed to Render

github.com/rohnshrma/backend-shopster

PART B

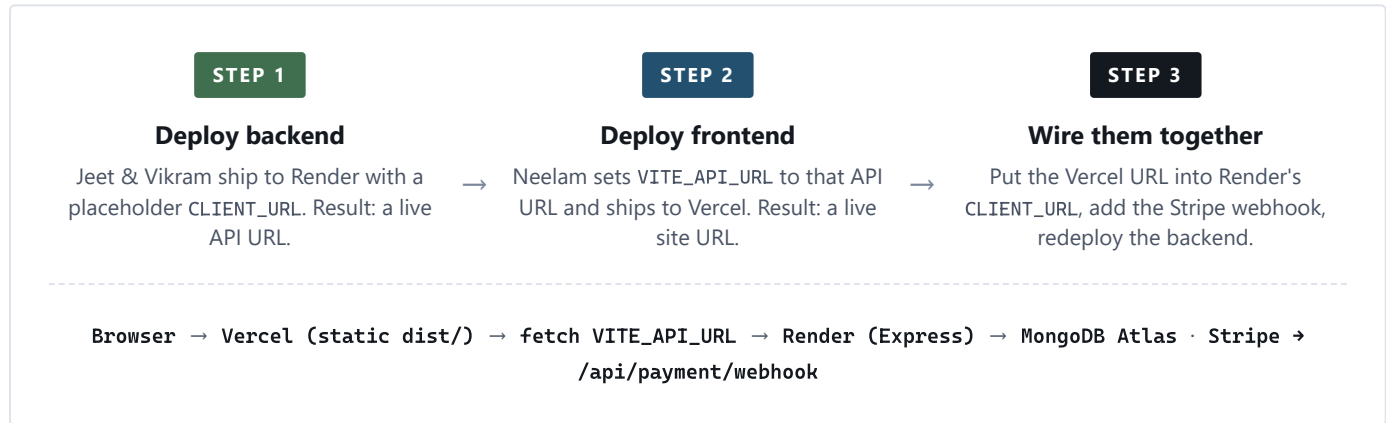
Neelam

Frontend — React 19 + Vite 8 SPA, deployed to Vercel

github.com/rohnshrma/frontend-shopster

Order of operations

The frontend needs the backend's URL, and the backend needs the frontend's URL. Neither exists until the other is deployed, so work in this order — do not try to do A and B fully in parallel.



Accounts and access everyone needs

WHAT	WHO NEEDS IT	NOTES
GitHub account with push access	All three	Both repos live under <code>rohnsharma</code> ; ask for collaborator access if you cannot push.
Render account	Jeet, Vikram	Sign up with GitHub so repo import is one click. Free tier is fine.
Vercel account	Neelam	Sign up with GitHub. Hobby plan is fine.
MongoDB Atlas account	Jeet, Vikram	Free M0 shared cluster.
Stripe account (test mode)	Jeet, Vikram	Needed for the secret key and the webhook signing secret.

Never commit secrets

`.env` is git-ignored in both repos and must stay that way. Every secret in this guide is typed into the Render or Vercel dashboard, never into a file that gets pushed. If a key ever lands in a commit, rotate it in Atlas/Stripe immediately — deleting the commit is not enough.

Backend on Render

Deploying backend-shopster — an Express 5 API using Mongoose 9, JWT auth, and Stripe Checkout — as a Render Web Service backed by MongoDB Atlas.

A1 Set up MongoDB Atlas

Render has no database of its own for this project; the API connects out to Atlas.

1. At `cloud.mongodb.com`, create a project and a free **M0** cluster. Pick a region close to the Render region you will choose in A4.
2. **Database Access** → **Add New Database User**. Username + a generated password, role *Read and write to any database*. Copy the password somewhere safe now — Atlas will not show it again.
3. **Network Access** → **Add IP Address** → **Allow access from anywhere** (0.0.0.0/0).
4. **Connect** → **Drivers** and copy the connection string. Replace `<password>` with the real password and add the database name before the `?`:

```
mongodb+srv://shopster_user:YOUR_PASSWORD@cluster0.xxxxx.mongodb.net/shopster?
retryWrites=true&w=majority
```

Why 0.0.0.0/0 is required here

Render's free instances do not have a fixed outbound IP address, so there is no single IP you could allow-list. The database user's password is what protects the cluster — make it long and random. If the password contains `@` `:` `/` `?` `#` `&`, URL-encode it (for example `@` becomes `%40`) or the connection string will parse wrong.

A2 Prepare the repository

Two small additions before the first deploy. Both go in the backend repo root.

Add a health-check route

`server.js` mounts routes only under `/api/*`, so hitting the bare domain returns `Cannot GET /`. That makes "is it up?" hard to answer and gives Render nothing cheap to ping. Add this just above `app.use("/api/product", ...)`:

```
app.get("/healthz", (req, res) => {
  res.status(200).json({ status: "ok", uptime: process.uptime() });
});
```

Pin the Node version

Mongoose 9 and Express 5 need a modern Node. Add an `engines` field to `package.json` so Render does not pick an older default:

```
"engines": {
  "node": ">=20.0.0"
},
```

Then confirm the app still starts locally and push:

```
npm install
npm start          # expect: Database connect successfully / Server Running On Port: 3000

git add .
git commit -m "chore: add health route and node engines for Render"
git push origin main
```

Already correct — do not change these

- "start": "node server.js" is exactly the start command Render needs. Do not point it at nodemon.
- `const PORT = process.env.PORT || 3000` already reads Render's injected port. Leave it alone.
- `/api/payment` is mounted *before* `express.json()` so Stripe's raw body survives for signature checking. Do not reorder that block.

A3 Create the Render Web Service

1. At `dashboard.render.com` choose **New** → **Web Service**.
2. Connect GitHub, then pick **rohnshrma/backend-shopster**.
3. Fill in the settings below and click **Create Web Service** — but expect the first build to fail health checks until A4 adds the environment variables. That is normal.

FIELD	VALUE
Name	shopster-backend — becomes <code>https://shopster-backend.onrender.com</code>
Language / Runtime	Node
Branch	main
Root Directory	<i>Leave blank</i> — the backend is its own repo, so <code>package.json</code> is already at the root
Build Command	<code>npm install</code>
Start Command	<code>npm start</code>
Instance Type	Free
Region	The one nearest your Atlas cluster (e.g. Singapore for an Asia cluster)
Health Check Path	<code>/healthz</code> — under Advanced; requires the route added in A2
Auto-Deploy	On — every push to <code>main</code> redeploys

A4 Add environment variables

In the service, open **Environment** → **Add Environment Variable** and add each row. These mirror `.env.example`.

KEY	VALUE	NOTES
MONGO_URI	The Atlas string from A1	Must include the <code>/shopster</code> database name.
JWT_SECRET	A long random string	Signs admin/seller tokens. Generate with <code>node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"</code> .
JWT_BUYER	A <i>different</i> long random string	Signs buyer tokens. Must not equal <code>JWT_SECRET</code> — that separation is what keeps a buyer token from being accepted on admin routes.
STRIPE_SECRET_KEY	<code>sk_test_...</code>	Stripe Dashboard → Developers → API keys. Use the secret key, not the publishable one.
STRIPE_WEBHOOK_SECRET	<code>whsec_placeholder</code> for now	Real value comes in A6, once the service has a URL.
CLIENT_URL	<code>http://localhost:5173</code> for now	Replaced with the Vercel URL in Part C. Drives CORS and Stripe redirect URLs.
NODE_VERSION	22	Optional but recommended; makes the runtime explicit rather than whatever Render defaults to.

Do not set PORT

Render injects `PORT` itself and routes traffic to it. If you add your own `PORT` variable, the app will listen on the wrong port, Render will never see it come up, and the deploy fails with *"No open ports detected"*. Leave `PORT` out of the dashboard entirely.

A5 Deploy and read the logs

Saving the environment variables triggers a redeploy. Open the **Logs** tab and watch for both lines:

```
Database connect successfully: cluster0-shard-00-01.xxxxx.mongodb.net
Server Running On Port: 10000
```

Then verify from your own machine (replace the host with your service URL):

```
curl https://shopster-backend.onrender.com/healthz
# {"status":"ok","uptime":12.4}

curl https://shopster-backend.onrender.com/api/product
# a JSON product list (an empty array is fine on a fresh database)
```

Free tier cold starts

A free Render service spins down after roughly 15 minutes with no traffic. The next request wakes it and can take **30–60 seconds**. That is not a bug — but it does mean the first page load after a quiet period looks broken. Tell Neelam so the frontend does not get blamed, and mention it in any demo. Upgrading to a paid instance removes the spin-down.

A6 Connect the Stripe webhook

Checkout only marks an order paid when Stripe calls back. That callback needs a public URL, which you now have.

1. Stripe Dashboard → **Developers** → **Webhooks** → **Add endpoint**.
2. Endpoint URL: `https://shopster-backend.onrender.com/api/payment/webhook`
3. Select events to listen to — at minimum `checkout.session.completed`. Add the other events your `paymentController` handles if you have extended it.
4. After creating it, click **Reveal** under *Signing secret* and copy the `whsec_...` value.
5. Back in Render, replace the placeholder `STRIPE_WEBHOOK_SECRET` with it and save. The service redeploys.
6. In Stripe, use **Send test webhook** and confirm a 200 response in the endpoint's delivery log.

If the webhook returns 400 "signature verification failed"

The signing secret is wrong or stale — each endpoint has its own, and a test-mode secret will not verify live-mode traffic. Re-copy it and redeploy. If it still fails, confirm nobody has moved the `app.use("/api/payment", paymentRoutes)` line below `express.json()` in `server.js`; a parsed body can no longer be signature-checked.

A7 Backend sign-off checklist

- ☐ `/healthz` returns 200 over HTTPS
- ☐ Logs show *Database connect successfully* with no reconnect loop
- ☐ `GET /api/product` returns JSON
- ☐ Register + login through `/api/auth` returns a token
- ☐ Stripe test webhook delivers 200
- ☐ All seven environment variables are set; `PORT` is *not* among them
- ☐ No `.env` file in the repo (`git ls-files | grep .env` shows only `.env.example`)
- ☐ The live API URL has been sent to Neelam

A8 Backend troubleshooting

SYMPTOM	CAUSE AND FIX
Deploy fails: <i>No open ports detected</i>	A <code>PORT</code> variable was set manually, or the start command was changed. Remove <code>PORT</code> ; confirm the start command is <code>npm start</code> .
<i>Fail to connect Database</i> then the process exits	Atlas is rejecting the connection. Check three things in order: Network Access has <code>0.0.0.0/0</code> ; the password in <code>MONGO_URI</code> is correct and URL-encoded; the database user exists with read/write role.
Build fails on <code>npm install</code>	Usually a Node version mismatch. Set <code>NODE_VERSION=22</code> and redeploy. Make sure <code>package-lock.json</code> is committed.
<code>ERR_MODULE_NOT_FOUND</code> at startup	A case-mismatched import. Windows filesystems are case-insensitive, Render's Linux is not — <code>./Router/x.js</code> works locally and fails there. Check the import path against the real filename.
First request of the day takes ~50s	Free-tier cold start. Expected. See the note in A5.
Stripe redirects land on <code>undefined/order-success/...</code>	<code>CLIENT_URL</code> is unset. It is interpolated directly into the Stripe <code>success_url</code> . Set it (Part C) and redeploy.

Frontend on Vercel

Deploying `frontend-shopster` — a React 19 single-page app built by Vite 8, talking to the Render API through a single fetch wrapper — as a Vercel static site.

Wait for Part A

You need the live backend URL from Jeet and Vikram before step B3. Steps B1 and B2 can be done now; B3 onward cannot. Do not deploy with a `localhost` API URL and plan to fix it later — Vite bakes the value into the built files, so a wrong value ships to real users.

B1 Verify the production build locally

Vercel runs the same build you can run on your laptop. Catch failures here, where the feedback loop is seconds instead of minutes.

```
cd frontend-shopster
npm install
npm run build      # must finish with no errors and create dist/
npm run preview    # serves dist/ at http://localhost:4173 - click through the app
```

If `npm run build` fails, fix it before touching Vercel — the deploy will fail the same way. A build that only fails in production is almost always a case-sensitive import path (`./components/Navbar` vs `./Components/Navbar`), because Vercel builds on Linux.

B2 Add `vercel.json` for client-side routing

`App.jsx` uses `BrowserRouter`, so routes like `/checkout` exist only in JavaScript — there is no `checkout.html` on disk. Without a rewrite rule, loading or refreshing any URL other than `/` returns Vercel's 404 page. Create `vercel.json` in the repo root:

```
{
  "rewrites": [
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

```
git add vercel.json
git commit -m "chore: add SPA rewrite config for Vercel"
git push origin main
```


Rewrite, not redirect

A rewrite serves `index.html` while the browser keeps the original URL, so React Router can read the path and render the right page. A redirect would change the address bar to `/` and drop the user on the home page instead.

B3 Import the project on Vercel

1. At `vercel.com/new`, choose **Import Git Repository** and pick **rohnshrma/frontend-shopster**.
2. Vercel detects Vite automatically. Confirm the settings below match — do not override them unless something is wrong.
3. **Do not click Deploy yet.** Expand *Environment Variables* first and do B4 in the same screen.

FIELD	VALUE
Framework Preset	Vite
Root Directory	<code>./</code> — the frontend is its own repo
Build Command	<code>npm run build</code>
Output Directory	<code>dist</code>
Install Command	<code>npm install</code>
Node.js Version	22.x (Settings → General, after import)

B4 Set VITE_API_URL

This is the one variable the frontend has, and the single most common thing to get wrong. It is read in `src/api/axios.js`:

```
const API_BASE_URL = import.meta.env.VITE_API_URL || "http://localhost:3000/api";
```

KEY	VALUE	ENVIRONMENTS
VITE_API_URL	<code>https://shopster-backend.onrender.com/api</code>	Production, Preview, Development

Three rules for this value

- **It must end in `/api`.** Every call is written as `API("/product")`, so the base carries the prefix. Without it every request 404s.
- **No trailing slash.** `.../api/` produces `.../api//product`, which Express does not match.
- **It must be `https://`.** The Vercel site is HTTPS; a plain `http://` API call is blocked by the browser as mixed content, silently, with no useful error on screen.

The variable is baked in at build time

Vite replaces `import.meta.env.VITE_API_URL` with a literal string while building. Changing the value in the Vercel dashboard does *nothing* to an already-built site. After any change you must **Deployments** → ... → **Redeploy** (with *Use existing Build Cache* unchecked). Also note the `VITE_` prefix is mandatory — Vite refuses to expose variables without it, and the value is publicly visible in the shipped bundle, so never put a secret here.

B5 Deploy

Click **Deploy**. The build takes a minute or two; watch the log for the Vite output and the *Build Completed* line. You get a production URL like `https://frontend-shopster.vercel.app`.

From now on every push to `main` redeploys production automatically, and every pull request gets its own preview URL.

B6 Verify in the browser

Open the production URL and check each of these, with DevTools open on the Network tab.

- ☐ Home page renders and the product list loads — confirms the API base URL is right
- ☐ In Network, requests go to `https://<render-url>/api/...` and return 200
- ☐ Navigate to an inner page, then **hard-refresh it** — still renders, no 404 (confirms `vercel.json`)
- ☐ Buyer register and login succeed; a token lands in `localStorage`
- ☐ Add to cart, then reload — cart survives
- ☐ Checkout redirects to Stripe's hosted page (needs Part C done)
- ☐ Console is free of CORS and mixed-content errors
- ☐ The layout holds on a phone-width viewport

If the very first load hangs for ~50 seconds

That is the Render free tier waking up, not a frontend fault. Give it a minute and retry before debugging. Worth handling in the UI later with a loading state on the first fetch.

B7 Frontend troubleshooting

SYMPTOM	CAUSE AND FIX
404 on refresh of any non-root route	<code>vercel.json</code> is missing, malformed, or not in the repo root. Confirm it is committed and pushed, then redeploy.
Every API call 404s	<code>VITE_API_URL</code> is missing the <code>/api</code> suffix, or has a trailing slash. Fix it and redeploy <i>without</i> the build cache.
Requests still hit <code>localhost:3000</code>	The variable was added after the build, so the old value is baked in. Redeploy with the cache cleared.
<i>Blocked by CORS policy</i> in the console	Backend problem, not yours. <code>CLIENT_URL</code> on Render must exactly equal your Vercel origin. See Part C.
Blank white page, console shows a bare module error	A runtime crash in a component. Reproduce with <code>npm run preview</code> locally, where the stack trace is readable.
Build fails on Vercel but passes locally	Case-sensitive import path, or a dependency in <code>devDependencies</code> that the build actually needs at runtime. Read the build log's first error, not the last.
Preview deployments fail auth	Each preview has its own URL, which <code>CLIENT_URL</code> does not cover. Test auth flows on the production URL, or ask for the preview origin to be added to CORS.

Wiring the two together

Both halves are live but do not yet trust each other. These last steps close the loop — they take about five minutes and must be done together.

C1 Point the backend at the real frontend

CLIENT_URL does two jobs in the backend: it is the CORS allow-list origin in `server.js`, and it is the prefix for Stripe's `success_url` and `cancel_url`. Until it matches the Vercel domain, the browser blocks every API call and Stripe redirects nowhere useful.

1. Neelam sends the exact production origin, for example `https://frontend-shopster.vercel.app`.
2. Jeet or Vikram updates CLIENT_URL in Render → Environment to that value and saves.
3. Wait for the automatic redeploy to reach *Live*.

The value must match character for character

`https://frontend-shopster.vercel.app` — scheme included, **no trailing slash**, no path. CORS compares origins as exact strings: a trailing `/`, an `http://`, or a `www.` that is not really there will all fail, and the browser error will just say the origin is not allowed.

C2 Confirm the Stripe webhook points at the live API

Re-check that the endpoint added in A6 is `https://<render-url>/api/payment/webhook` and that its signing secret matches STRIPE_WEBHOOK_SECRET in Render. Run one **Send test webhook** and confirm a 200.

C3 Run one full end-to-end pass

Do this on the live Vercel URL, in a fresh browser profile or private window, with all three of you watching.

- ☐ Register a new buyer, log out, log back in
- ☐ Browse products, open a detail page, add two items to the cart
- ☐ Place an order and reach Stripe's hosted checkout page
- ☐ Pay with the test card 4242 4242 4242 4242, any future expiry, any CVC
- ☐ Land back on `/order-success/<id>` — proves CLIENT_URL is correct
- ☐ The order shows as paid in the buyer's order list — proves the webhook fired and was verified
- ☐ Stripe's webhook log shows a 200 for that event
- ☐ Log in as admin and see the order in the admin list
- ☐ Nothing red in the browser console throughout

C4 Who owns what after launch

AREA	OWNER	WHERE TO LOOK FIRST
API errors, 500s, database issues	Jeet & Vikram	Render → Logs
Payments and webhook failures	Jeet & Vikram	Stripe → Developers → Webhooks → delivery log
UI bugs, routing, build failures	Neelam	Vercel → Deployments → build log; browser console
CORS errors	Both	Symptom is in the browser; the fix is <code>CLIENT_URL</code> on Render
Environment variable changes	Whoever owns that side	Backend: redeploy is automatic. Frontend: redeploy manually, cache cleared

C5 Routine maintenance

- **Rotate secrets** if a key is ever pasted into chat, a screenshot, or a commit. Both JWT secrets and the Stripe key can be regenerated and swapped in Render without code changes — note that rotating a JWT secret logs everyone out.
- **Going live with real payments** means swapping to Stripe live keys *and* creating a second webhook endpoint in live mode with its own signing secret. Test-mode secrets will not verify live traffic.
- **Custom domain:** add it in Vercel → Settings → Domains, then immediately update `CLIENT_URL` on Render to the new origin, or CORS breaks the moment people start using it.
- **Rollback:** Vercel keeps every deployment — *Promote to Production* on an older one is instant. Render offers a rollback to a previous deploy from the service's Events tab.

Shopster deployment guide · v1.0 · 3 September 2026 — Backend: Express 5, Mongoose 9, Stripe 18 on Render · Frontend: React 19, Vite 8, React Router 7 on Vercel. Dashboard layouts change; the settings values in this guide are what matter, not the exact position of the buttons.